

STEMFX: LEARNING MIXING STYLE REPRESENTATIONS VIA AUTOREGRESSIVE FX CHAIN PREDICTION ON SOURCE-SEPARATED STEMS

Yuan-Chiao Cheng¹ Jui-Te Wu¹ Brian Chen¹
 Yen-Tung Yeh² Yu-Hua Chen² Yi-Hsuan Yang²
¹ Independent Researcher ² National Taiwan University, Taiwan

{im31132, bryan051003, chenbrian0314, affige}@gmail.com, {f12942179, f08946011}@ntu.edu.tw

ABSTRACT

Audio mixing style encompasses the artistic and technical decisions a mix engineer makes, including level balancing, spatialization, and the choice, ordering, and parameterization of audio effects (FX) on each stem. FX chains are a key determinant of this style, yet existing approaches to modeling them remain limited. Some operate on stereo mixtures without explicit per-stem FX chain modeling, others fix the number or type of effects per track, and many require differentiable effect implementations or scarce multitrack datasets. We present STEMFX, a framework that learns mixing style representations by autoregressively predicting variable-length FX chains on source-separated stems. A Transformer decoder predicts tokenized FX chains autoregressively, while a band-split multi-band CNN encoder with FiLM conditioning captures per-stem spectral structure. To enable large-scale paired training, we extract pseudo-stems from about 105K songs via source separation and augment them using MultiAFx, a toolkit unifying 85 audio effects from 7 Python libraries. Evaluated on mixing style retrieval, STEMFX outperforms all baseline models across all tested chain lengths. On paired mixing style transfer, STEMFX achieves the best spectral fidelity and the highest listener preference, over 4000 times faster than iterative optimization.

1. INTRODUCTION

A mixing engineer applies a distinct chain of audio effects to each instrument track: an equalizer to brighten the vocals, a compressor to tighten the drums, reverb to place the guitar in a room. The collective choice of effects, their ordering, and their parameter settings across all stems defines the *mixing style*. An FX chain can be thought of as a sentence: just as language derives expressiveness from an open vocabulary and flexible length, a faithful representation of mixing style should support variable-length

chains with arbitrary effects. Learning such representations would improve mixing style recognition, retrieval, and transfer.

Yet prior approaches severely constrain this vocabulary. Contrastive encoders such as FxEncoder [1] require large batch sizes to provide sufficient negative pairs, increasing GPU memory cost. Differentiable mixing approaches [2, 3, 4, 5] fix the number and type of effects per track and require differentiable implementations, restricting the set of usable effects. ST-ITO [6] can optimize effect chains unseen during training, but only at inference time, costing thousands of forward passes and, in our experiments, over 1000 seconds per example; its AFx-Rep encoder learns effect representations through a classification pretext task over a fixed inventory of effect presets. Most methods also require paired dry/wet multitrack stems, limiting them to small datasets: MUSDB18 [7] with 150 tracks, MedleyDB [8] with about 200, and MoisesDB [9] with 240.

We propose STEMFX, which treats FX chain prediction as a sequence generation problem. By predicting tokenized FX chains autoregressively on source-separated stems, STEMFX removes the constraints of fixed chain lengths and effect ordering. Unlike DDSP methods [4, 5], it needs no differentiable implementations, predicting discrete token sequences rather than optimizing audio-domain losses; unlike methods that parameterize a fixed set of effect presets [2], it predicts the effect structure and the chain length themselves. The output is human-readable and interpretable, e.g., “sox_compend $\rightarrow$ threshold=-20 dB, ratio=4:1 $\rightarrow$ sox_reverb $\rightarrow$ decay=1.5 s,” allowing engineers to inspect, modify, and execute the predicted chain. Both the STEMFX codebase, with source code and trained model weights, and the MultiAFx library are released as open source.¹

Our contributions are as follows:

1. **STEMFX Framework.** To the best of our knowledge, the first framework to jointly train an audio encoder and decoder end-to-end for learning mixing style representations through autoregressive FX chain generation. This differs from LLM-based tool-calling ap-



© Y.-C. Cheng, J.-T. Wu, B. Chen, Y.-T. Yeh, Y.-H. Chen, and Y.-H. Yang. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Y.-C. Cheng, J.-T. Wu, B. Chen, Y.-T. Yeh, Y.-H. Chen, and Y.-H. Yang, “StemFX: Learning Mixing Style Representations via Autoregressive FX Chain Prediction on Source-Separated Stems”, in *Proc. of the 27th Int. Society for Music Information Retrieval Conf.*, Abu Dhabi, UAE, 2026.

¹ <https://github.com/barry-mir/stemfx> and <https://github.com/barry-mir/multiafx>

proaches [10], which keep the audio encoder frozen and fine-tune only the adapter and LLM via LoRA, so their generation objective never shapes the audio encoder.

2. **BSFiLM Encoder.** A band-split multi-band CNN encoder [11] with Feature-wise Linear Modulation (FiLM) conditioning [12] from hand-crafted mixing features, bringing band-split processing to mixing style representation learning.
3. **Sep-Aug Pipeline and MultiAFx Toolkit.** Source separation applied to a large mixture dataset creates pseudo-stems, which are augmented with random chains drawn from MultiAFx, our pip-installable wrapper over 85 effects from 7 Python libraries. Together they yield paired training data at about 105K-song scale, orders of magnitude larger than existing methods. See Section 3.7 for details.

2. RELATED WORK

2.1 Automatic Mixing and Style Transfer

Automatic mixing systems transfer the sonic characteristics of a reference mix to new material, through either representation learning or direct parameter prediction. FxEncoder [1] disentangles mixing style from content via contrastive learning on multitrack audio, its successor Fx-Encoder++ [13] refines the architecture, and MEGAMI [14] builds on these representations for conditional diffusion-based mixing. Differentiable console methods such as Diff-MST [2] and the work of Steinmetz et al. [3] backpropagate through fixed effect chains, while ST-ITO [6] optimizes effect parameters at inference time. LLM2Fx-Tools [10] is closest to our formulation, using autoregressive sequence modeling to generate the FX chain, but it keeps the audio encoder frozen and fine-tunes only a language model adapter via LoRA, so the generation objective does not improve the audio representations. Collectively, these methods either omit explicit per-stem FX chain prediction, fix the effect structure, or rely on inference-time optimization. STEMFX sidesteps these constraints by processing separated stems at scale and jointly training encoder and decoder for variable-length FX chain prediction.

2.2 Audio Representation Learning and Band-Split Architectures

Self-supervised and contrastive audio representations such as CLAP [15], COLA [16], CLMR [17], MERT [18], Music2Latent [19], and HTS-AT [20] target content-level tasks and do not capture fine-grained mixing characteristics. Band-split architectures are effective for source separation [21, 22, 23, 24] and multi-band CNNs for acoustic classification [11], but have not been applied to mixing style representation. We adapt this paradigm, using per-band CNNs to capture frequency-specific characteristics such as EQ curves and compression.

3. METHOD

3.1 Framework Overview and Problem Formulation

Figure 1 illustrates the STEMFX architecture. The framework jointly trains a BSFiLM Encoder and a Transformer-based FX Chain Generator end-to-end. The encoder maps both the original and augmented stems into conditioning vectors that summarize the mixing style difference, and the generator autoregressively predicts the tokenized FX chain that was applied. By treating FX chain prediction as a sequence generation problem, STEMFX supports variable-length chains with arbitrary effects.

We formalize the task as follows. Let $\mathbf{x} \in \mathbb{R}^{8 \times T}$ denote a set of four source-separated stereo stems (vocals, bass, drums, other) at 44.1 kHz. Given an original input $\mathbf{x}^{\text{orig}}$ and a target $\mathbf{x}^{\text{aug}}$ augmented with a known FX chain, the goal is to predict the per-stem FX chain $\mathcal{F}$ that transforms $\mathbf{x}^{\text{orig}}$ to match $\mathbf{x}^{\text{aug}}$. Each stem’s chain is an ordered list of effect–parameter pairs:

$$\mathcal{F}_s = [(f_1, \theta_1), (f_2, \theta_2), \dots, (f_{N_s}, \theta_{N_s})], \quad s \in \mathcal{S} \quad (1)$$

where $\mathcal{S} = \{\text{vocals, bass, drums, other}\}$, f_i is a specific effect name (e.g., `sox_comband`), θ_i its parameters (e.g., threshold, ratio), and N_s varies per stem. We concatenate the 4 per-stem FX chains into the full chain $\mathcal{F} = \{\mathcal{F}_s\}_{s \in \mathcal{S}}$, which is serialized into a single token sequence $\mathbf{y}$ for autoregressive prediction (Section 3.3).

3.2 BSFiLM Encoder

The encoder maps the 8-channel input $\mathbf{x}$ to a mixing style embedding $\mathbf{e} \in \mathbb{R}^{512}$, following a multi-band CNN design inspired by Kim et al. [11] and adapted for representation learning with FiLM conditioning. Each of the 8 channels is transformed into a log-mel spectrogram with $n_{\text{fft}} = 2048$, $\text{hop} = 512$, and $n_{\text{mels}} = 80$, yielding shape $(B, 8, 80, T')$. The frequency axis is divided into overlapping sub-bands of 20 mel bins with 10-bin overlap. Each sub-band is processed by an independent 2-layer CNN with 7×7 kernels, $32 \rightarrow 64$ channels, batch normalization, and max pooling, producing per-band feature maps.

FiLM [12] injects 64 hand-crafted mixing features into each CNN layer, as detailed in Table 1. These features are adapted from the Diff-MST [2] mixing feature set to our four-stem setting, computing dynamics, spectral, stereo, and inter-stem descriptors per stem. A 2-layer MLP generates scale γ and shift β parameters:

$$\mathbf{x}' = \gamma \odot \mathbf{x} + \beta, \quad (\gamma, \beta) = \text{MLP}(\mathbf{f}) \quad (2)$$

The per-band feature maps are concatenated along the channel axis, flattened across frequency, and temporally pooled via learned attention weights to produce the 512-dimensional embedding (1.79M parameters total). Independent per-band CNNs allow each sub-band to specialize in detecting frequency-specific processing such as band-limited EQ, compression, or de-essing.

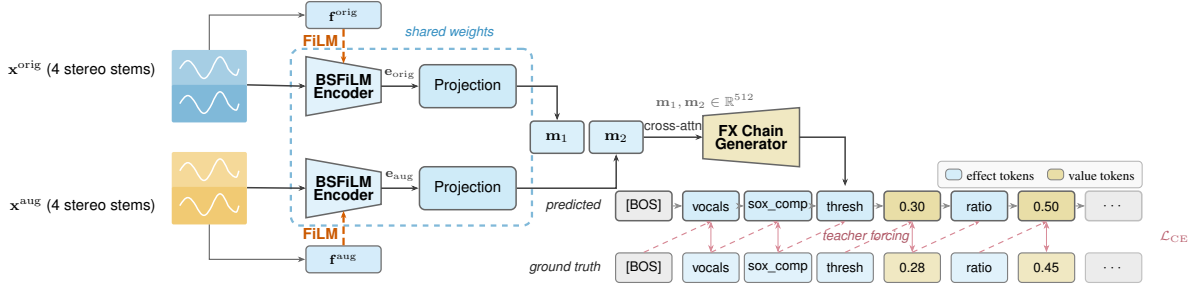


Figure 1. STEMFX system overview. Original and target stems pass through the shared BSFiLM Encoder to produce embeddings, projected into conditioning vectors $\mathbf{m}_1, \mathbf{m}_2$ that the Transformer decoder attends to via cross-attention. The FX Chain Generator is trained with teacher forcing and cross-entropy loss against the next ground-truth token.

Table 1. Hand-crafted mixing features for FiLM conditioning (64 total).

Category	Feature (formula)	/Stem	Total
Dynamics	RMS: $\sqrt{\frac{1}{T} \sum x_c^2} (\times 2 \text{ ch.})$	6	24
	Crest: $20 \log_{10}(\frac{\max x_c }{\text{RMS}_c}) (\times 2 \text{ ch.})$		
	K-weighted loudness ($\times 2 \text{ ch.}$): $-0.691 + 10 \log_{10}(\sum_c G_c \cdot \text{RMS}_c^2)$		
Spectral	Band energy: $\bar{E}_{\text{low}}, \bar{E}_{\text{mid}}, \bar{E}_{\text{high}}$	5	20
	Tilt: $\rho(\text{bin index}, \bar{E}_{\text{mel}})$		
	Flatness: $\exp(\log \bar{S}) / \bar{S}$		
Stereo	ILD: $20 \log_{10}(\text{RMS}_L / \text{RMS}_R)$	3	12
	Correlation: $\sum \tilde{L} \tilde{R} / \ \tilde{L}\ \ \tilde{R}\ $		
	Mid-side ratio: $E_{\text{side}} / E_{\text{mid}}$		
Rel. loud.	$L_{\text{stem}} - L_{\text{mix}}$	1	4
Masking	$\sigma(-(E_s - \max_{j \neq s} E_j))$	1	4

x_c : waveform of channel c ; S : mel spectrogram; $\bar{E}$: mean energy over time and channels; $\tilde{L}, \tilde{R}$: zero-mean left/right channels; $E_{\text{mid}} = ((L+R)/2)^2$, $E_{\text{side}} = ((L-R)/2)^2$; σ : sigmoid; E_s : mel energy of stem s ; G_c : channel weighting per ITU-R BS.1770.

3.3 FX Chain Tokenization

We serialize variable-length, multi-stem FX chains into flat token sequences. The vocabulary of 358 tokens comprises 4 special tokens, 4 stem tokens, 85 effect name tokens, 164 parameter name tokens, and 101 quantized value bins. An example sequence is:

```
[BOS] vocals sox_gain sox_gain_gain_db value_55
sox_compand sox_compand_threshold_db value_30
sox_compand_ratio value_50 ... [SEP] [EOS]
```

Continuous parameter values are normalized to $[0, 1]$ and quantized to 101 bins. Frequency-domain parameters such as filter cutoffs use log-scale normalization to match perceptual sensitivity, $\hat{v} = (\log v - \log v_{\min}) / (\log v_{\max} - \log v_{\min})$, while all other parameters such as gain, ratio, and attack/release times use linear normalization, $\hat{v} = (v - v_{\min}) / (v_{\max} - v_{\min})$. The vocabulary is extensible; adding a new effect requires only a name token, parameter tokens, and normalization ranges.

3.4 FX Chain Generator

Both $\mathbf{x}^{\text{orig}}$ and $\mathbf{x}^{\text{aug}}$ pass through the shared BSFiLM Encoder to produce 2 embeddings $\mathbf{e}^{\text{orig}}$ and $\mathbf{e}^{\text{aug}}$. Each embedding is independently projected via a shared projection

head consisting of LayerNorm, Linear, ReLU, and Linear layers, producing conditioning vectors $\mathbf{m}_1, \mathbf{m}_2 \in \mathbb{R}^{512}$ that serve as key-value entries for the decoder’s cross-attention.

Our paired-embedding design is inspired by AFx-Rep [6], which encodes both original and augmented audio to learn effect representations from their paired embeddings. We adopt this paired-input paradigm but replace the classification head with a 6-layer Transformer decoder with $d_{\text{model}} = 512$, 8 heads, and FFN dimension 2048. The decoder attends to $[\mathbf{m}_1, \mathbf{m}_2]$ via cross-attention and projects to the full vocabulary via a linear head, predicting all token types in a unified output space. Cross-attention separates audio conditioning from the token sequence, allowing the decoder to query the style information independently at each layer. An alternative is to prepend $\mathbf{m}_1, \mathbf{m}_2$ as prefix tokens. We leave the comparison of conditioning strategies to future work.

3.5 Training Objective

The ground-truth FX chain $\mathcal{F}$ is serialized into a token sequence $\mathbf{y} = (y_1, \dots, y_L)$. The decoder is trained with teacher forcing, minimizing cross-entropy at each step.

$$\mathcal{L} = \mathcal{L}_{\text{CE}} = - \sum_t \log p(y_t | y_{<t}, \mathbf{m}_1, \mathbf{m}_2) \quad (3)$$

where $y_{<t}$ denotes the ground-truth prefix.

The decoder applies a causal mask so that each position attends only to preceding tokens and the conditioning vectors $\mathbf{m}_1, \mathbf{m}_2$. Training uses AdamW with learning rate 3×10^{-4} , weight decay 0.01, linear warmup over 1000 steps, cosine decay, batch size 64, and gradient clipping at norm 1.0. We train jointly for 60 epochs, approximately 56 GPU hours, on a single NVIDIA RTX 5090 GPU using 10-second audio clips. The full model comprises the BSFiLM Encoder with 1.79M parameters and the 6-layer Transformer decoder with 26.2M parameters, totaling 28.0M parameters.

3.6 Inference

At inference time, given a pair of audio inputs (original and target), the encoder produces conditioning vectors $\mathbf{m}_1, \mathbf{m}_2$ that summarize the mixing style difference. The decoder then autoregressively generates the full token sequence via greedy decoding, predicting effect names, pa-

parameter names, and quantized values in a single pass. Unlike generative models that aim to produce a range of plausible variants, our goal is to recover the chain that comes as close as possible to the ground truth, so we use greedy decoding instead of sampling. The predicted tokens are parsed back into a structured FX chain specification, mapping each stem to its ordered effect sequence with associated parameter values.

3.7 Sep-Aug Pipeline

We propose the Sep-Aug Pipeline, which applies source separation to a large mixture dataset and then augments the resulting pseudo-stems with known FX chains to generate paired training data at scale, lifting the multitrack bottleneck described in Section 1.

The pipeline has three stages. First, a pretrained source separation model splits each song into four pseudo-stems (vocals, bass, drums, other). Second, *cross-song stem mixing* draws each stem in a training example from a different song, decorrelating musical content from mixing characteristics and preventing the encoder from relying on song-level cues such as key or tempo. Third, each stem is independently augmented with random effects from MultiAFx, with a macro-category constraint preventing consecutive effects from the same category. The result is paired data with complete parameter metadata for supervised training. See Section 4.1 for more details.

MultiAFx Toolkit. We introduce MultiAFx, a Python library wrapper unifying seven backends [25, 26, 27, 28, 29, 30, 31] behind a consistent interface with 85 effects across 12 macro-categories. The library supports chain generation with macro-category constraints, parameter range definitions with linear/log normalization, and fault-tolerant batch processing.

4. EXPERIMENTS

4.1 Setup

Training Data. We apply the Sep-Aug Pipeline to the open FMA dataset [32] using SCNet [22] for source separation, producing about 105K songs with 4 pseudo-stems each. Each song is augmented with 1–10 random effects per stem from MultiAFx. Because FMA contains predominantly mono or stereo mixtures rather than true multitracks, many songs lack meaningful energy in one or more separated stems. We measure integrated loudness for every stem and exclude those below -40 dB. Cross-song stem mixing does not prevent the encoder from observing inter-stem mixing relationships: the encoder still receives all four stems jointly and can learn how their relative levels, spectral balance, and spatial placement interact, regardless of whether they originate from the same song. However, cross-song mixing may produce disharmonious combinations, such as stems in different keys or tempos. We leave the investigation of musical coherence effects on learned representations to future work.

Evaluation Data. For all experiments we use MUSDB18 [7], which provides professionally mixed

multitracks not seen during training by any model. To avoid evaluating on tracks with near-silent stems, we select the most balanced 10-second window per track by computing per-stem RMS energy and requiring every stem to contribute at least 10% of total energy. Stems are then FX-normalized following Martinez-Ramirez et al. [33] to a common baseline before augmentation with *pedalboard* [34] effects at chain lengths of 1–8. *pedalboard* is not included in MultiAFx and was not used by any of the baseline methods during training, ensuring that all systems are evaluated on out-of-domain audio effects. FX normalization is used only to construct evaluation pairs. It is not applied during training, where the original and augmented views of a pseudo-stem already differ by exactly the chain to be predicted, so normalizing to a common baseline is unnecessary.

4.2 Mixing Style Retrieval

We evaluate whether the learned embeddings capture mixing style rather than musical content. Following the protocol of Fx-Encoder++ [1, 13], both the query and the candidate pool consist of augmented stem sets. The task is to retrieve, from a pool of $N = 500$ candidates, the one that shares the same FX chain as the query but contains different musical content. Retrieval is performed by cosine similarity between embeddings.

Baselines. We compare against three external encoders using released pretrained weights. **AFx-Rep** [6] uses PANNs with mid/side input and an effect classification pretext task. **Fx-Encoder++** [13] is a contrastive stereo encoder producing 2048-d embeddings. **CLAP** [15] uses HTSATtiny with audio-text contrastive learning and 512-d embeddings. These baselines differ in training data and augmentation, so performance gaps cannot be attributed to the objective alone. To control for this, we additionally train **BSFiLM-CL**, the BSFiLM Encoder trained with NT-Xent contrastive learning [35] on the same Sep-Aug data. Following the Fx-Encoder++ paradigm, positive pairs share the same FX chain applied to different content, but we omit the instrument-level query mechanism since BSFiLM processes separated stems directly. This isolates the training objective as the sole variable between STEMFX and BSFiLM-CL. We do not include a comparable controlled experiment for AFx-Rep’s classification objective, as the Sep-Aug Pipeline generates over 100K distinct FX chain variants, making closed-set classification impractical at this scale. Figure 2 reports Top-1, Top-5, Top-10 accuracy and Mean Reciprocal Rank (MRR). STEMFX outperforms all baselines across all FX counts and all metrics. At 1 effect, the task is most challenging because subtle parameter differences must be distinguished. STEMFX achieves 16.8% Top-1 and 0.235 MRR versus 3.0% Top-1 and 0.067 MRR for AFx-Rep. At 8 effects, STEMFX reaches 86.8% Top-1 and 0.903 MRR versus 68.4% and 0.754 for AFx-Rep and 38.0% and 0.459 for Fx-Encoder++. The BSFiLM-CL comparison reveals two insights. First, BSFiLM-CL reaches 77.8% Top-1 at 8 effects, 9.0 percentage points below STEMFX (86.8%), demonstrating that

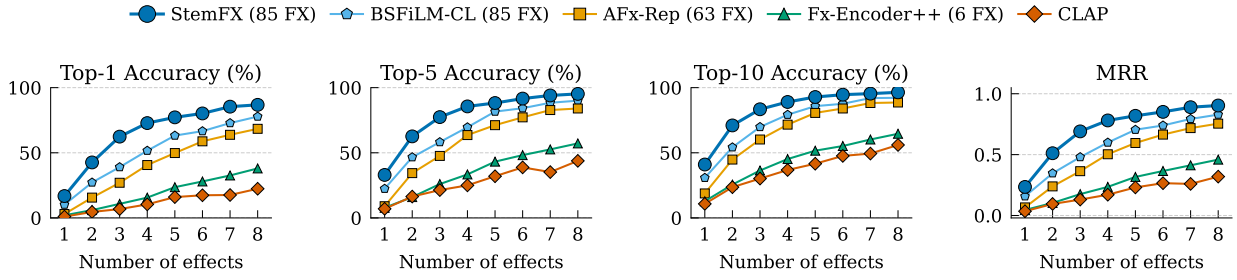


Figure 2. Mixing style retrieval vs. number of applied effects: Top-1, Top-5, Top-10 accuracy (%) and Mean Reciprocal Rank (MRR). STEMFX outperforms all baselines across all chain lengths and metrics, including BSFiLM-CL (same architecture and data, contrastive training). Numbers in parentheses denote the number of effects each model was trained with.

Table 2. Paired mixing style transfer results. MUSHRA score is the mean rating from 20 listeners on a 0–100 similarity scale. STEMFX forced and both ITO methods share the same fixed signal chain.

Method	Synth.	Real Mix		Time
	MRSTFT ↓	MRSTFT ↓	MUSHRA ↑	
STEMFX	2.35	1.44	60.6	0.24s
STEMFX forced	3.12	2.10	25.3	1.91s
ITO + AFx-Rep	3.71	2.53	30.6	1033s
ITO + BSFiLM	2.89	2.06	19.9	1717s
Target mix	0.00	0.00	96.6	—
FX-normalized	3.38	1.49	54.9	—

under matched architecture and data conditions, the FX chain prediction objective produces stronger mixing representations than contrastive learning. Second, BSFiLM-CL substantially outperforms Fx-Encoder++ despite sharing the same contrastive training paradigm, indicating that the BSFiLM Encoder combined with the Sep-Aug Pipeline yields a stronger foundation than Fx-Encoder++’s architecture and training data.

4.3 Paired Mixing Style Transfer

Given a paired original and target, STEMFX can transfer the target’s mixing style by predicting and executing the FX chain that bridges them. We evaluate on two test sets. The *Synthetic* set contains 20 MUSDB18 pairs where targets are augmented with 8 `pedalboard` effects per stem, out-of-domain from any training data, stress-testing generalization across a wide range of devices and parameter values. The *Real Mix* set takes FX-normalized stems as input and the original professionally mixed MUSDB18 tracks as the target, testing whether methods can recover mixing decisions in real-world scenarios. We conduct a MUSHRA-style listening test with 20 listeners with music production experience on the Real Mix set only. Listeners rate how similar each condition sounds to the target mix on a 0–100 scale in a blind, randomized setting. Each song includes the four method outputs alongside two hidden references: the target mix itself as the high anchor and the FX-normalized input as the low anchor. The Synthetic set is excluded from listening because up to eight randomly combined effects produce unnaturally dense processing, making perceptual quality judgments difficult. We measure spectral fidelity via multi-resolution STFT loss,

MRSTFT, and inference time. Table 2 reports the results.

We compare four methods. **STEMFX** uses autoregressive generation with default settings, predicting both which effects to apply and their parameters. All three fixed-chain methods operate on the same universal signal chain covering gain staging, highpass/lowpass filters, four parametric EQ bands, compression, distortion, chorus, delay, and reverb. Each device can be effectively bypassed with neutral parameter values, so the chain spans styles from minimal processing to complex multi-effect configurations. **STEMFX forced** predicts only value tokens for this fixed chain, with structural tokens injected from the chain template. The two ITO baselines follow the inference-time optimization approach of ST-ITO [6], optimizing the same universal chain via CMA-ES with population 64, $\sigma_0=0.3$, and 25 iterations. Each of the four stems is processed independently. **ITO + AFx-Rep** uses AFx-Rep [6] cosine similarity as the optimization objective, while **ITO + BSFiLM** substitutes the BSFiLM Encoder.

Synthetic set. STEMFX achieves the best MRSTFT of 2.35 at 0.24s per example, over 4000 times faster than iterative optimization. The forced variant enables a controlled comparison with ITO, as all three share the same signal chain and parameter space, isolating feedforward prediction from iterative search. Neither ITO variant beats STEMFX, indicating that selecting the right effects with proper order matters more than perfectly optimizing parameters of a predetermined chain.

Real Mix set. On professional mixes, the advantage of free generation becomes even more pronounced. STEMFX achieves the best MRSTFT of 1.44 and the highest MUSHRA score of 60.6, the only method that surpasses the FX-normalized low anchor of 54.9. The high anchor scores 96.6, confirming that listeners can reliably identify the target mix. All fixed-chain methods score 19.9 to 30.6. Listening to low-scoring cases confirms that these methods frequently apply audible distortion or modulation where the target mix uses neither. The universal chain exposes effects that many target mixes do not use, and driving every such effect to neutral parameters is difficult. ITO must jointly optimize all parameters in the chain without a prior for which effects to bypass, and STEMFX forced likely suffers from distribution mismatch since the training data rarely contains neutral-parameter instances. STEMFX avoids both failure modes by selecting only the effects that match the target, a fundamental advantage of the autore-

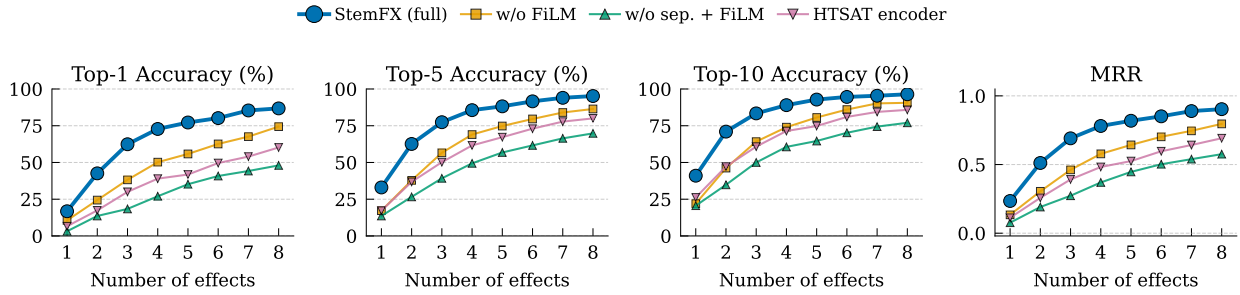


Figure 3. Ablation study: retrieval performance vs. number of applied effects for each model variant. STEMFX (full) is the proposed model; each variant removes one component.

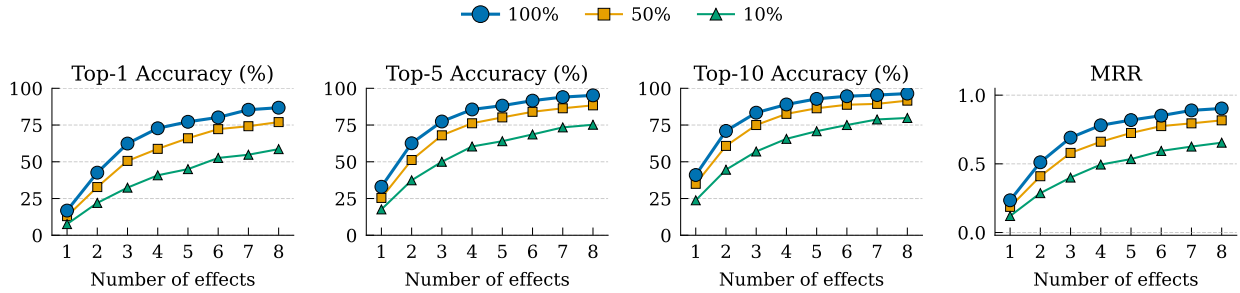


Figure 4. Dataset scale analysis: retrieval vs. number of applied effects at training set sizes of 10%, 50%, and 100% of 105K songs. The Sep-Aug Pipeline enables training at a scale orders of magnitude beyond existing paired datasets.

gressive formulation which chooses *which* effects to apply, not only *how* to parameterize a fixed set. We also demonstrate unpaired style transfer on the demo page.²

4.4 Ablation Study

Figure 3 isolates the contribution of each STEMFX component using three ablation variants. Removing FiLM conditioning reduces Top-1 from 86.8% to 74.4% at 8 effects, confirming that hand-crafted mixing features provide useful inductive bias. Replacing source-separated stems with stereo mixture input causes the largest degradation, dropping Top-1 to 48.0%. This variant also removes FiLM conditioning, since the hand-crafted features in Table 1 are per-stem and undefined for mixture input. The gap between w/o FiLM at 74.4% and w/o sep. + FiLM at 48.0% isolates source separation, showing that per-stem input accounts for 26.4 percentage points of improvement even without FiLM. Replacing the BSFiLM Encoder with HTSAT-tiny [20] drops Top-1 to 60.2%, showing that the stem-aware multi-band CNN design outperforms general audio transformers for mixing style representation.

4.5 Dataset Scale Analysis

Figure 4 shows retrieval accuracy as a function of training set size (10%, 50%, 100% of 105K songs). Performance improves consistently from 58.6% at 10% size to 77.0% at 50% size and 86.8% at 100% size, demonstrating that the Sep-Aug Pipeline’s ability to scale training data translates directly into better representations. This scaling behavior is not achievable with real multitrack datasets, which are limited to hundreds of songs.

²<https://barry-mir.github.io/stemfx-demo/>

5. LIMITATIONS AND FUTURE WORK

STEMFX can only predict effects contained in the FX set it was trained on, and supporting additional effects requires retraining on data that includes them. STEMFX also operates on the four stems produced by current separation systems, a step beyond estimating mixing style from the stereo mixture alone, though extending it to the finer decompositions of a professional session is left to future work. Pseudo-stems additionally inherit the errors of the separation model, and we do not quantify how these artifacts propagate into the learned representation; a comparison against a model trained on clean multitrack stems would isolate this effect. Finally, training chains are sampled at random, whereas mix engineers apply structured, genre-dependent processing. Whether the representation captures such conventions beyond random effect compositions remains open.

6. CONCLUSION

We presented STEMFX, a framework for learning mixing style representations through autoregressive FX chain prediction on source-separated stems, with three contributions: an end-to-end autoregressive formulation that outperforms contrastive learning under matched conditions, 86.8% vs. 77.8% Top-1; the BSFiLM Encoder, whose FiLM conditioning and per-stem input contribute 12.4% and 26.4%; and the Sep-Aug Pipeline with the MultiAFx Toolkit, which scales paired training data to about 105K songs and lifts Top-1 from 58.6% to 86.8% as the training set grows.

7. REFERENCES

- [1] J. Koo, M. A. Martinez-Ramirez, W.-H. Liao, S. Uhlich, K. Lee, and Y. Mitsufuji, “Music mixing style transfer: A contrastive learning approach to disentangle audio effects,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023. [Online]. Available: <https://arxiv.org/abs/2211.02247>
- [2] S. S. Vanka, C. J. Steinmetz, J.-B. Rolland, J. D. Reiss, and G. Fazekas, “Diff-MST: Differentiable mixing style transfer,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.08889>
- [3] C. J. Steinmetz, J. Pons, S. Pascual, and J. Serrà, “Automatic multitrack mixing with a differentiable mixing console of neural audio effects,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021. [Online]. Available: <https://arxiv.org/abs/2010.10291>
- [4] C. J. Steinmetz, N. J. Bryan, and J. D. Reiss, “Style transfer of audio effects with differentiable signal processing,” *Journal of the Audio Engineering Society*, vol. 70, no. 9, pp. 708–721, 2022. [Online]. Available: <https://arxiv.org/abs/2207.08759>
- [5] C. J. Steinmetz, “dasp-pytorch: Differentiable audio signal processors in PyTorch,” <https://github.com/csteinmetz1/dasp-pytorch>, 2023.
- [6] C. J. Steinmetz, S. Singh, M. Comunità, I. Ibnyahya, S. Yuan, E. Benetos, and J. D. Reiss, “ST-ITO: Controlling audio effects for style transfer with inference-time optimization,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2024, best Paper Award. [Online]. Available: <https://arxiv.org/abs/2410.21233>
- [7] Z. Rafii, A. Liutkus, F.-R. Stöter, S. I. Mimilakis, and R. Bittner, “MUSDB18 – a corpus for music separation,” <https://sigsep.github.io/datasets/musdb.html>, 2017.
- [8] R. M. Bittner, J. Salamon, M. Tierney, M. Mauch, C. Cannam, and J. P. Bello, “MedleyDB: A multi-track dataset for annotation-intensive MIR research,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2014.
- [9] I. Pereira, F. Araújo, F. Korzeniowski, and R. Vogl, “MoisesDB: A dataset for source separation beyond 4-stems,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.15913>
- [10] S. Doh, J. Koo, M. A. Martinez-Ramirez, W. Choi, W.-H. Liao, Q. Wu, J. Nam, and Y. Mitsufuji, “LLM2Fx-Tools: Tool calling for music post-production,” in *Proc. International Conference on Learning Representations (ICLR)*, 2026. [Online]. Available: <https://arxiv.org/abs/2512.01559>
- [11] D. Kim, S. Park, D. K. Han, and H. Ko, “Multi-band CNN architecture using adaptive frequency filter for acoustic event classification,” *Applied Acoustics*, vol. 172, p. 107579, 2021.
- [12] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proc. AAAI Conference on Artificial Intelligence*, 2018. [Online]. Available: <https://arxiv.org/abs/1709.07871>
- [13] Y.-T. Yeh, J. Koo, M. A. Martinez-Ramirez, W.-H. Liao, Y.-H. Yang, and Y. Mitsufuji, “Fx-Encoder++: Extracting instrument-wise audio effects representations from mixtures,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.02273>
- [14] E. Moliner, M. A. Martinez-Ramirez, J. Koo, W.-H. Liao, K. W. Cheuk, J. Serrà, V. Välimäki, and Y. Mitsufuji, “Automatic music mixing using a generative model of effect embeddings,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026. [Online]. Available: <https://arxiv.org/abs/2511.08040>
- [15] B. Elizalde, S. Deshmukh, M. Al Ismail, and H. Wang, “CLAP: Learning audio concepts from natural language supervision,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023. [Online]. Available: <https://arxiv.org/abs/2206.04769>
- [16] A. Saeed, D. Grangier, and N. Zeghidour, “Contrastive learning of general-purpose audio representations,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021. [Online]. Available: <https://arxiv.org/abs/2010.10915>
- [17] J. Spijkervet and J. A. Burgoyne, “Contrastive learning of musical representations,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2021. [Online]. Available: <https://arxiv.org/abs/2103.09410>
- [18] Y. Li, R. Yuan, G. Zhang, Y. Ma, X. Chen, H. Yin, C. Xiao, C. Lin, A. Ragni, E. Benetos, N. Gyenge, R. Dannenberg, R. Liu, W. Chen, G. Xia, Y. Shi, W. Huang, Z. Wang, Y. Guo, and J. Fu, “MERT: Acoustic music understanding model with large-scale self-supervised training,” in *Proc. International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2306.00107>
- [19] M. Pasini, S. Lattner, and G. Fazekas, “Music2Latent: Consistency autoencoders for latent audio compression,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.06500>

- [20] K. Chen, X. Du, B. Zhu, Z. Ma, T. Berg-Kirkpatrick, and S. Dubnov, “HTS-AT: A hierarchical token-semantic audio transformer for sound classification and detection,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022. [Online]. Available: <https://arxiv.org/abs/2202.00874>
- [21] Y. Luo and J. Yu, “Music source separation with band-split RNN,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 1893–1901, 2023. [Online]. Available: <https://arxiv.org/abs/2209.15174>
- [22] W. Tong, J. Zhu, J. Chen, S. Kang, T. Jiang, Y. Li, Z. Wu, and H. Meng, “SCNet: Sparse compression network for music source separation,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.13276>
- [23] W.-T. Lu, J.-C. Wang, Q. Kong, and Y.-N. Hung, “Music source separation with band-split RoPE transformer,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 481–485. [Online]. Available: <https://arxiv.org/abs/2309.02612>
- [24] J.-C. Wang, W.-T. Lu, and M. Won, “Mel-band RoFormer for music source separation,” *arXiv preprint arXiv:2310.01809*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.01809>
- [25] L. Narskog and C. Bagwell, “SoX – sound exchange,” <https://sox.sourceforge.net/>, 2005.
- [26] I. Jordal, “audiomentations: A Python library for audio data augmentation,” <https://github.com/iver56/audiomentations>, 2020.
- [27] Y.-Y. Yang, M. Hira, Z. Ni, A. Chourdia, A. Astafurov, C. Chen, C.-F. Yeh, C. Puhersch, D. Pollack, D. Genzel *et al.*, “Torchaudio: Building blocks for audio and speech processing,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022. [Online]. Available: <https://arxiv.org/abs/2110.15018>
- [28] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.
- [29] B. McFee, C. Raffel, D. Liang, D. P. Ellis, M. McVicar, E. Battenberg, and O. Nieto, “librosa: Audio and music signal analysis in Python,” in *Proc. 14th Python in Science Conference*, 2015, pp. 18–24.
- [30] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [31] C. J. Steinmetz and J. D. Reiss, “pyloudnorm: A simple yet flexible loudness meter in Python,” in *150th Audio Engineering Society Convention*, 2021.
- [32] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, “FMA: A dataset for music analysis,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2017. [Online]. Available: <https://arxiv.org/abs/1612.01840>
- [33] M. A. Martinez-Ramirez, W.-H. Liao, G. Fabbro, S. Uhlich, C. Nagashima, and Y. Mitsufuji, “Automatic music mixing with deep learning and out-of-domain data,” in *Proc. International Society for Music Information Retrieval Conference (ISMIR)*, 2022. [Online]. Available: <https://arxiv.org/abs/2208.11428>
- [34] P. Sobot, “Pedalboard: A python library for adding effects to audio,” <https://github.com/spotify/pedalboard>, 2021.
- [35] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *Proc. International Conference on Machine Learning (ICML)*, 2020.